

Online supplement:
A Comparative Analysis of Relational Database
Access-Layer
Performance in Express.js for PostgreSQL and
MySQL

Mateusz Miotk

Faculty of Mathematics, Physics and Informatics, University of Gdańsk
`mateusz.miotk@ug.edu.pl`

This supplement carries measurement detail referenced from the main text. All tables are generated by the replication package (<https://github.com/mmioTk/express-db-access-performance>; archived at <https://doi.org/10.5281/zenodo.21359919>) from the same raw data as the main-text tables.

S1 Dispersion on MySQL

The main text reports the coefficient-of-variation table for PostgreSQL and the per-engine maxima. Table S1 gives the full per-layer, per-pattern CV on MySQL across the 25 independent replicates (maximum 6.6%, `knex`/insert).

Table S1: Coefficient of variation (%) of throughput across the 25 independent replicates, per access layer and pattern on MySQL. Low values indicate stable measurements.

Layer	<code>point_read</code>	<code>range_scan</code>	<code>deep_fetch</code>	<code>aggregation</code>	<code>write</code>
<code>mysql2</code>	3.2	2.7	2.2	1.8	6.6
<code>knex</code>	2.4	3.0	2.2	2.6	6.6
<code>drizzle</code>	3.0	2.2	2.4	2.2	5.2
<code>prisma</code>	4.2	2.0	1.0	4.0	5.7
<code>sequelize</code>	2.2	2.0	1.9	2.1	5.9
<code>typeorm</code>	2.5	1.9	3.0	2.5	6.0
<code>objection</code>	3.3	3.4	2.5	2.5	5.9
<code>mikroorm</code>	3.5	2.9	3.0	1.9	4.9

S2 Idiomatic round-trip counts

Table S2 reports, for each layer, the number of SQL statements the idiomatic deep fetch issues per request, captured from server-side statement logs. The counts motivate the main text’s observation that round-trip count alone does not determine throughput: the two layers issuing four statements (Prisma, Objection) sit at opposite ends of the throughput ladder.

Table S2: Number of SQL round trips each access layer issues per request on PostgreSQL, captured from server-side statement logging. The native drivers, Knex, and Drizzle issue a two-query deep fetch; the data-mapper ORMs’ eager-loading strategies choose their own counts. Full generated SQL and EXPLAIN plans are in the replication package.

Layer	Point read	Range scan	Deep fetch	Aggregation	Insert
pg	1	1	2	1	1
knex	1	1	2	1	1
drizzle	1	1	2	1	1
prisma	1	1	4	1	1
sequelize	1	1	1	1	1
typeorm	1	1	2	1	2
objection	1	1	4	1	1
mikroorm	1	1	1	1	1

S3 Write throughput under default versus relaxed durability

Table S3 gives the per-layer insert throughput under each engine’s default durability configuration (the paper’s primary write regime) and under the symmetric relaxed regime (secondary, 10 replicates), with the relaxed÷default ratio quoted in the main text.

S4 Equal-CPU-budget control

Table S4 gives the deep-fetch throughput with the server confined to 1, 2, or 4 cores (database and load generator pinned to disjoint cores); all nine values are quoted in the main text.

S5 Application-tier versus end-to-end CPU efficiency

Table S5 reports, per layer on the deep/nested fetch, the median application-process-tree CPU and database-server CPU, the combined core-count, and

Table S3: Insert throughput (req/s) under the default durability configuration (PostgreSQL `fsync=on`, `synchronous_commit=on`; MySQL `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`; median of 25 replicates) and under the relaxed regime (all of the above off; median of 10), with the relaxed÷default ratio. At 50 concurrent connections, group commit amortizes the per-commit flushes: relaxing durability buys at most 14% on PostgreSQL (and $\leq 4\%$ on the layer-bound ORMs) and 19–24% on MySQL.

Layer	PostgreSQL			MySQL		
	default	relaxed	ratio	default	relaxed	ratio
pg	6064	6895	1.14	–	–	–
mysql2	–	–	–	809	966	1.19
pg-tuned	6280	7150	1.14	–	–	–
mysql2-tuned	–	–	–	808	972	1.20
knex	4693	5212	1.11	795	948	1.19
drizzle	4230	4675	1.11	796	950	1.19
prisma	5465	5675	1.04	714	879	1.23
sequelize	2668	2653	0.99	772	925	1.20
typeorm	2595	2675	1.03	732	886	1.21
objection	3926	4245	1.08	787	944	1.20
mikroorm	1571	1607	1.02	663	822	1.24

Table S4: Deep-fetch throughput (req/s, PostgreSQL, median of 3) with the server process confined by `taskset` to 1, 2, or 4 cores (database and load generator pinned to disjoint cores). `pg` reaches full throughput on a single core; Prisma requires roughly four to approach it, quantifying the CPU subsidy behind its near-native throughput.

Layer	1 core	2 cores	4 cores
pg	3663	3277	3700
prisma	896	1726	3025
mikroorm	463	577	542

throughput per application-CPU-second and per combined (application+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database: `pg` leads application-tier efficiency by roughly $5\times$ over Prisma (3,741 vs. 740 req per app-CPU-s) but only $1.4\times$ end-to-end (813 vs. 596 req per combined-CPU-s), because it pushes about 3.6 database cores of work that the application-only metric ignores.

Table S5: Application-tier versus end-to-end CPU efficiency on the deep/nested fetch (PostgreSQL): median CPU (% of one logical core) of the Node.js process tree and of the database server, the combined core-count, and throughput per application-CPU-second and per combined (app+database) CPU-second. The application-only figure understates the compute of layers that delegate more work to the database; the combined figure is the end-to-end view. Load-generator CPU is excluded.

Layer	app CPU%	db CPU%	comb. cores	req/app-CPU-s	req/comb-CPU-s
pg	100	360	4.60	3741	813
knex	100	245	3.45	2491	722
drizzle	100	205	3.05	2080	682
prisma	498	120	6.18	740	596
sequelize	100	50	1.50	989	659
typeorm	105	120	2.25	1264	590
objection	100	50	1.50	1037	691
mikroorm	100	45	1.45	571	394

S6 Coordinated-omission-corrected open-loop sweep

Table S6 gives the full constant-arrival sweep (PostgreSQL, five layers, offered rates 250–4,000 requests/s) behind the open-loop paragraph of the main text: achieved throughput, corrected latency percentiles measured from each request’s *intended* send time, and timeout counts (10 s cap, clipped into the distribution).

S7 Pool-size sensitivity

The primary campaign fixes every layer’s connection pool at 10 to isolate the access layer from pool tuning. Table S7 varies the pool from 1 to 50 for a representative layer of each tier on the deep fetch (PostgreSQL, 50 connections). The three-band ordering (pg/Prisma, then Knex, then MikroORM) is preserved at every pool size, and each layer’s throughput saturates at a pool well below 50, so the fixed-pool choice does not drive the ranking; a very small pool (1–2) is the one regime that compresses the differences, because every layer then queues on connection acquisition.

Table S6: Open-loop tail latency, corrected for coordinated omission: p99 (ms) on the deep/nested fetch (PostgreSQL) under a constant offered request rate, with every latency measured from the request’s intended start time and timed-out requests included at their clipped value. [†]marks cells that did not sustain the offered rate (achieved<95% or any timeouts).

Layer	250/s	500/s	1000/s	2000/s	4000/s
pg	2	2	2	2	1201 [†]
prisma	3	2	2	4	2110 [†]
knex	2	2	2	33	9420 [†]
sequelize	2	3	41	10757 [†]	10944 [†]
mikroorm	3	15	11220 [†]	11399 [†]	11878 [†]

Table S7: Deep-fetch throughput (req/s, PostgreSQL, 50 connections, median of 3) as the connection pool varies from 1 to 50 for a representative layer of each tier. The primary campaign fixes the pool at 10; the three-band ordering (native and query-engine ORM on top, the query builder in the middle, the data-mapper ORM below) is preserved at every pool size, and each layer saturates at a pool well below 50, so the fixed-pool choice does not drive the ranking.

Layer	1	2	5	10	20	50
pg	895	1732	3203	3672	3764	3697
knex	750	1423	2455	2499	2648	2577
prisma	949	1670	3446	3351	3651	3565
mikroorm	535	539	548	570	584	587

S8 Transactional multi-statement write

Beyond the single-row insert of the primary matrix, Table S8 reports a transactional write, `POST /threads`, which inserts a post and five comments in one transaction through each layer’s transaction facility, for a representative layer of each tier on PostgreSQL under default durability. The full layer $\times$ engine matrix for this pattern is left to future work. Because the transaction commits once, its per-commit fsync dominates and the layers cluster much more tightly than the between-layer spread on reads; the ordering tracks the single-row insert, so the write conclusions extend to a multi-statement transactional write.

Table S8: Transactional write throughput (req/s) and tail latency (p99, ms) for `POST /threads`, which inserts a post and five comments in a single transaction, on PostgreSQL under default durability (50 connections, median of 5). A representative layer of each tier is shown; the full matrix is left to future work. Between-layer spread is $3.3\times$ (Prisma 2412 down to MikroORM 740), well below the $6.55\times$ read spread, and the ordering tracks the single-row insert, so the write conclusions extend to a multi-statement transactional write.

Layer	req/s	p99 (ms)
pg	2347	44
knex	2029	49
prisma	2412	41
typeorm	1790	48
mikroorm	740	90

S9 Dispersion on PostgreSQL

Table S9 gives the full per-layer, per-pattern coefficient of variation of throughput on PostgreSQL across the 25 independent replicates (maximum 7.6%, pg/insert), the companion to the MySQL table in Section S1.

S10 Resource footprint

Table S10 gives the per-layer application-process CPU (median % of one logical core) and peak resident memory on the deep/nested fetch (PostgreSQL), the detail behind the CPU trade-off discussed in the main text (Prisma $\approx 498\%$, the other layers $\approx 100\%$; peak RSS 204–271 MB).

Table S9: Coefficient of variation (%) of throughput across the 25 independent replicates, per access layer and pattern on PostgreSQL. Low values indicate stable measurements.

Layer	point_read	range_scan	deep_fetch	aggregation	write
pg	4.3	2.9	2.6	1.9	7.6
knex	2.6	3.1	3.2	3.8	5.5
drizzle	2.5	2.6	2.9	2.8	3.6
prisma	3.6	3.4	3.6	4.3	4.3
sequelize	3.1	2.5	2.2	1.7	3.8
typeorm	2.8	2.6	2.0	2.6	3.8
objection	3.9	3.0	2.9	2.7	3.4
mikroorm	2.7	2.4	2.9	2.9	3.7

Table S10: Server-process resource footprint on the deep/nested fetch (PostgreSQL): median CPU utilization (% of one core) and peak resident memory (MB) of the Node.js process under load. The load generator and database run in separate processes.

Layer	Category	CPU (%)	RSS (MB)
pg	native-driver	100	205
knex	query-builder	100	204
drizzle	orm-lightweight	100	250
prisma	orm	498	266
sequelize	orm	100	248
typeorm	orm	105	271
objection	orm	100	211
mikroorm	orm	100	255

S11 Pinned versions

Table S11 lists the pinned versions of every evaluated access layer and tool, recorded from the logfile on the measurement date.

Table S11: Pinned versions of the evaluated access layers and tooling.

Layer / tool	Version	Layer / tool	Version
pg	8.22.0	sequelize	6.37.8
mysql2	3.22.5	typeorm	0.3.30
knex	3.3.0	objection	3.1.5
drizzle-orm	0.36.4	@mikro-orm/core	6.6.15
@prisma/client	5.22.0	express	4.22.2
autocannon	7.15.0	PostgreSQL / MySQL	18.4 / 9.7.1

S12 Per-pattern detail: the read patterns

The two cheapest read patterns show the smallest between-layer spread and are summarized in the main text; their full throughput and p99 tables are given here. Table S12 reports the point read (primary-key lookup) and Table S13 the keyset range scan, each by access layer and engine. The consequential patterns (deep/nested fetch, aggregation, insert) are tabulated in the main text.

Table S12: Point read: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

Layer	Category	PostgreSQL		MySQL	
		req/s	p99	req/s	p99
pg	native-driver	6695	9.0	–	–
mysql2	native-driver	–	–	4337	16.0
pg-tuned	native-tuned	6851	9.0	–	–
mysql2-tuned	native-tuned	–	–	4302	14.0
knex	query-builder	5096	12.0	3837	15.0
drizzle	orm-lightweight	4555	13.0	3168	20.0
prisma	orm	6051	12.0	5993	11.0
sequelize	orm	3532	16.0	2746	21.0
typeorm	orm	4331	14.0	3517	16.0
objection	orm	4070	15.0	3222	18.0
mikroorm	orm	2185	26.0	1839	31.0

Table S13: Keyset range scan: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

Layer	Category	PostgreSQL		MySQL	
		req/s	p99	req/s	p99
pg	native-driver	3782	18.0	–	–
mysql2	native-driver	–	–	3231	21.0
pg-tuned	native-tuned	3811	17.0	–	–
mysql2-tuned	native-tuned	–	–	3216	18.0
knex	query-builder	3163	19.0	2952	20.0
drizzle	orm-lightweight	2905	20.0	2265	28.0
prisma	orm	3963	17.0	3936	18.0
sequelize	orm	2426	24.0	2074	27.0
typeorm	orm	2546	25.0	2474	24.0
objection	orm	2669	22.0	2514	23.0
mikroorm	orm	773	74.0	729	79.0

S13 Same-SQL control: full table

Table S14 gives the full same-SQL (raw-path) control on the deep fetch for both engines: throughput when every layer executes the identical control statement (verified byte-identical by server-side capture). The main text reports the resulting collapse toward parity and interprets it as a bound on the raw-execution component of the idiomatic deficit.

S14 Per-pattern detail: aggregation

Table S15 gives the full aggregation-pattern throughput and p99 by access layer and engine. RQ3 finds aggregation the least layer-sensitive pattern once every layer runs the same raw correlated-subquery plan; the main text reports the ranking and the leading figures.

S15 Idiomatic deep-fetch choices per access layer

Table S16 documents, for each access layer, the exact eager-loading API used on the deep/nested fetch, the number of SQL round trips it issues, the documented alternative loading strategy we did *not* use, and the query-logging state. Every layer uses its documented default (idiomatic) path; query logging is disabled and verified on all eleven adapters, and no lifecycle hooks or validation run on the read path. The six data-mapper and ORM layers split both ways on the join-versus-select-in default: Sequelize, TypeORM, and MikroORM default to a single join, while Prisma and Objection default to

Table S14: Bounding the raw-execution component of the deep-fetch spread: throughput (req/s) of the idiomatic deep fetch versus the *same-SQL* control (median of 10 replicates), in which every layer executes the identical two-statement SQL with identical row mapping through its raw-SQL facility. The ratio (idiomatic $\div$ same-SQL) measures the gap between each layer’s idiomatic relational-fetch path and the common raw-SQL path, a composite of query strategy, query count, protocol, the raw versus ORM API, and result marshalling and hydration changed together; the residual same-SQL spread (native-relative $1.47\times$ on PostgreSQL, $1.27\times$ on MySQL) bounds the raw-execution component.

Layer	PostgreSQL			MySQL		
	idiom.	same-SQL	ratio	idiom.	same-SQL	ratio
pg	3741	3643	1.03	–	–	–
mysql2	–	–	–	2405	2419	0.99
knex	2491	2906	0.86	2050	2305	0.89
drizzle	2080	3446	0.60	1446	2105	0.69
prisma	3685	4342	0.85	3701	4220	0.88
sequelize	989	2472	0.40	893	1912	0.47
typeorm	1327	2974	0.45	1170	2517	0.46
objection	1037	2845	0.36	860	2243	0.38
mikroorm	571	2869	0.20	535	2172	0.25

Table S15: Aggregation: throughput (req/s, higher is better) and tail latency (p99, ms, lower is better) by access layer and engine.

Layer	Category	PostgreSQL		MySQL	
		req/s	p99	req/s	p99
pg	native-driver	7035	10.0	–	–
mysql2	native-driver	–	–	4195	17.0
pg-tuned	native-tuned	7731	9.0	–	–
mysql2-tuned	native-tuned	–	–	4194	17.0
knex	query-builder	5754	12.0	3945	17.0
drizzle	orm-lightweight	6427	11.0	3944	18.0
prisma	orm	7893	9.0	7383	9.0
sequelize	orm	4807	15.0	3390	19.0
typeorm	orm	6127	12.0	4371	17.0
objection	orm	3775	18.0	2963	22.0
mikroorm	orm	5279	14.0	3590	19.0

separate batched queries. The loading-strategy sensitivity check reported in the main text (Table S18) switches the join-default layers to select-in where that alternative is a byte-identical drop-in. Full per-adapter detail is in the replication package’s `METHODOLOGY.md`.

Table S16: Idiomatic deep-fetch implementation per access layer: the eager-loading API actually used, the SQL round trips it issues (measured from server-side statement logging for the portable layers; by construction for the native variants), the documented alternative loading strategy we did *not* use, and the query-logging state. Query logging is disabled everywhere; no lifecycle hooks or validation run on the read path (details in the replication package’s `METHODOLOGY.md`).

Layer	Deep-fetch API	Round-trips	Alternative not used	Logging
pg	hand-written JOIN $\times 2$	2	n/a (hand-written SQL)	off
mysql2	hand-written JOIN $\times 2$	2	n/a (hand-written SQL)	off
pg-tuned	named-prepared JOIN $\times 2$	2	n/a (hand-written SQL)	off
mysql2-tuned	execute() JOIN $\times 2$	2	n/a (hand-written SQL)	off
knex	builder .join() $\times 2$	2	n/a (hand-written SQL)	off
drizzle	builder .innerJoin() $\times 2$	2	relational query API (db.query)	off
prisma	include:	4	relationLoadStrategy: 'join'	off
sequelize	include: (separate:false)	1	separate:true (select-in)	off
typeorm	relations:	2	relationLoadStrategy: 'query'	off
objection	.withGraphFetched()	4	.withGraphJoined()	off
mikroorm	populate:	1	strategy: 'select-in'	off

S16 Open-loop tail latency on MySQL

Table S17 is the MySQL companion to the PostgreSQL open-loop sweep (Supplement Table S6): coordinated-omission-corrected p99 on the deep/nested fetch at a constant offered rate. The sub-saturation tail is flat and small on both engines, and each layer collapses once the offered rate crosses its capacity, so the intrinsic-tail ordering holds on MySQL as on PostgreSQL.

S17 Alternative eager-loading sensitivity

Table S18 reports the loading-strategy sensitivity check: for the data-mapper ORMs whose alternative strategy is a byte-identical drop-in, idiomatic deep-fetch throughput versus the alternative, on both engines. Both endpoints were verified to return byte-identical responses before timing.

S18 MySQL insert commit-flush mechanism

Table S19 gives the direct `performance_schema` evidence behind the MySQL insert bottleneck: sampling the redo-log and binary-log flush wait instruments around a fixed insert workload at default durability, the flush is a

Table S17: Open-loop tail latency on **MySQL**, corrected for coordinated omission: p99 (ms) on the deep/nested fetch under a constant offered request rate, every latency measured from the intended start time and timed-out requests clipped into the distribution. [†]marks cells that did not sustain the offered rate. As on PostgreSQL (Supplement Table S6), the sub-saturation tail is flat and small and each layer collapses once the offered rate crosses its capacity, so the intrinsic-tail ordering is engine-robust.

Layer	250/s	500/s	1000/s	2000/s	4000/s
mysql2	2	2	4	14	9658 [†]
prisma	3	3	3	4	2006 [†]
knex	2	2	2	297	10900 [†]
sequelize	3	3	2334 [†]	10814 [†]	10945 [†]
mikroorm	4	34	11269 [†]	11673 [†]	11800 [†]

Table S18: Alternative eager-loading sensitivity on the deep/nested fetch: idiomatic throughput (req/s) versus the alternative loading strategy, both engines, for the layers whose alternative is a byte-identical drop-in. Switching Sequelize and MikroORM from their idiomatic single join to select-in makes them slower on both engines (0.68–0.75 $\times$), so their deep-fetch deficit is not an artifact of a poor default. Objection is the converse: switching its idiomatic batched select-in to a single join (**withGraphJoined**) is byte-identical only on MySQL, where it is 1.6 $\times$ faster, so part of Objection’s MySQL deficit *is* a default-strategy choice. **typeorm** is omitted (its query strategy errors on the ordered nested **findOne** in the pinned version, itself a portability caveat), as is Objection on PostgreSQL, where **withGraphJoined** did not return a byte-identical result. The idiomatic column is re-measured in the same paired run (five replicates) as its alternative, so its absolute values differ from Table ?? by day-to-day drift; the idiomatic-versus-alternative ratio is the comparison of interest.

Layer	Switch	PostgreSQL		MySQL	
		idiom.	alt.	idiom.	alt.
sequelize	join→select-in	897	668	788	590
mikroorm	join→select-in	421	296	423	289
objection	select-in→join	–	–	802	1277

shared engine floor—similar per-insert across the native driver, query builder, and ORM despite their very different read performance—and relaxing durability removes it.

Table S19: MySQL insert commit-flush mechanism (`performance_schema` wait instruments, default durability `innodb_flush_log_at_trx_commit=1`, `sync_binlog=1`): per-insert wall time, the redo-log+binlog flush wait per insert, and the flush share of wall time, over 4000 inserts per layer. The flush wait is a shared engine floor – similar across layers despite their very different read performance – so it caps insert throughput regardless of the access layer. Relaxing durability (`mysql2`, `trx_commit=0`, `sync_binlog=0`) removes the flush and lifts throughput to 787 req/s, confirming the floor.

Layer (MySQL)	req/s	per-insert (ms)	flush/insert (ms)	flush share
<code>mysql2</code>	670	1.493	0.588	39%
<code>knex</code>	671	1.489	0.571	38%
<code>mikroorm</code>	588	1.699	0.686	40%
<i>Relaxed durability (control):</i>				
<code>mysql2*</code>	787	1.27	0.011	1%

S19 Post-restart environmental-robustness check

Table S20 reports the environmental-robustness check: after a full restart of both database engines, a representative deep-fetch subset re-measured eight days after the primary campaign. The layer ranking reproduces on both engines (`pg` > Prisma $\gg$ MikroORM on PostgreSQL; Prisma > `mysql2` $\gg$ MikroORM on MySQL); absolute throughput is 3–23% lower, the day-to-day host drift the paper’s relative analysis is designed to absorb. Because PostgreSQL and MySQL cells are not measured simultaneously, this drift is non-uniform across engines (3% on `mysql2`, 13% on `pg`, 23% on MikroORM/PostgreSQL), so the cross-engine interaction ratios could in principle carry some residual drift; they are far larger than 23% (reads 1.0–1.6, writes 2.4–7.7), so the engine-dependence conclusion holds.

Table S20: Environmental-robustness check (review 6.7): after a full restart of both database engines (cold buffer pools, fresh connections), deep-fetch throughput of a representative subset re-measured eight days after the primary campaign, against the primary median. The layer ranking reproduces on both engines; absolute throughput drifts with host conditions (larger on PostgreSQL), which is exactly the day-to-day variation the paper’s *relative* analysis is designed to absorb. Single host, so this checks temporal robustness, not cross-machine generalization.

Engine	Layer	primary req/s	post-restart req/s	ratio
PostgreSQL	pg	3741	3270.5	0.87
PostgreSQL	prisma	3685	3211	0.87
PostgreSQL	mikroorm	571	437.5	0.77
MySQL	mysql2	2405	2331	0.97
MySQL	prisma	3701	3161	0.85
MySQL	mikroorm	535	430.5	0.81